

《计算机组成原理》实验报告

年级、专业、班级	2024 级计算机科学与技术 03、02、01	姓名	严浩睿、吴林兴、霍秋明
实验题目	实验五 Cache 设计与实现		
实验时间	2026 年 5 月 31 日	实验地点	DS1421
实验成绩		实验性质	☐ 验证性 ☒ 设计性 ☐ 综合性
教师评价： ☒算法/实验过程正确； ☒源程序/实验内容提交； ☒程序结构/实验步骤合理； ☒实验结果正确； ☒语法、语义正确； ☒报告规范； 其他： 评价教师：			
实验目的 1. 加深对 Cache 原理的理解,包括地址映射、替换算法、写策略等核心概念。 2. 使用 Verilog 实现 Cache 模块,加深对状态机设计的理解。 3. 理解写直达(Write-Through)与写回(Write-Back)两种写策略的区别与实现差异。 4. 掌握直接映射 Cache 的写回 + 写分配策略,以及字节/半字/字写掩码合并方法。			

报告完成时间: 2026 年 6 月 1 日

1 实验内容

本实验要求在给定的 MIPS/SoC 环境中用 Verilog 实现 Cache 模块。实验包给出了完整的 MIPS CPU、类 SRAM 到 AXI 的转换桥、RAM、外设和仿真环境，需要补全指令 Cache(i_cache)和数据 Cache(d_cache)两个核心模块。

具体实现要求如下：

1. 参考指导书中直接映射写直达 Cache 的实现，将数据 Cache 改为写回 (Write-Back) + 写分配 (Write-Allocate) 策略，并通过仿真测试。
2. 指令 Cache 采用直接映射，只处理取指读请求，缺失时从下一级存储器补充。
3. Cache 容量为 4KB，每行 1 个 32 位 word，共 1024 行。32 位地址拆分为 20 位 tag + 10 位 index + 2 位 offset。
4. 接口采用类 SRAM 握手协议 (req / addr_ok / data_ok)，CPU 侧和内存侧均遵循此协议。
5. 代码需通过 Vivado 行为仿真测试和板级自测验证。

2 实验设计

2.1 总体架构

Cache 模块位于 CPU 和下一级存储器之间，作为 CPU 取指和数据访存的缓存层。整体架构如下：

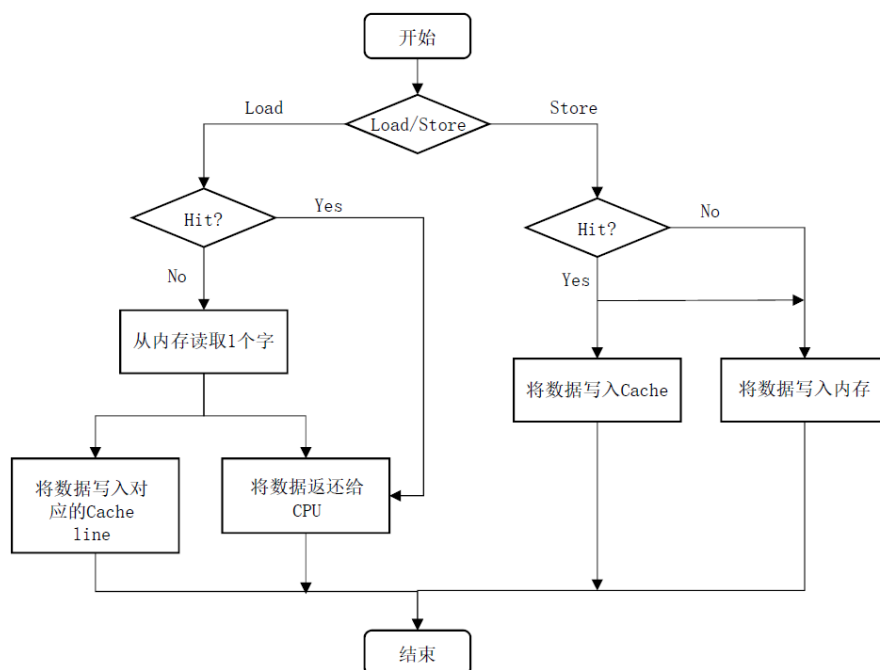


图 1: Cache 总体架构

顶层 `cache.v` 作为总封装, 内部实例化两个子模块:

- **i_cache**: 处理 CPU 取指请求, 只读, 缺失时从下一级指令存储器读入。
- **d_cache**: 处理 CPU 数据访问 (`load/store`), 写回 + 写分配策略, 维护 `dirty` 位。

2.2 地址划分

32 位 CPU 地址拆分为 `tag`、`index`、`offset` 三部分:

字段	位宽	地址范围	含义
tag	20 bit	addr[31:12]	内存块标识
index	10 bit	addr[11:2]	Cache 行号 (1024 行)
offset	2 bit	addr[1:0]	字节偏移 (4 字节/字)

表 1: 地址划分

命中判断条件:

```
hit = cache_valid[index] && (cache_tag[index] == tag)
```

先用 `index` 定位到唯一一行, 再检查该行 `valid` 位是否为 1 且 `tag` 是否与请求地址的 `tag` 相等。

2.3 模块一: 指令 `Cache(i_cache.v)`

2.3.1 功能描述

指令 `Cache` 只处理 CPU 取指读请求。命中时直接从 `Cache` 数组返回指令; 缺失时进入 `S_REFILL` 状态, 向下一级指令存储器发读请求, 数据返回后填入 `Cache` 并返回 CPU。

2.3.2 接口定义

表 2: 指令 Cache 接口定义

信号名	方向	位宽	功能描述
clk	Input	1	时钟信号
rst	Input	1	复位信号,高有效
cpu_inst_req	Input	1	CPU 取指请求
cpu_inst_addr	Input	32	取指地址
cpu_inst_rdata	Output	32	返回给 CPU 的指令
cpu_inst_addr_ok	Output	1	地址握手: Cache 接收取指地址
cpu_inst_data_ok	Output	1	数据握手: 指令数据有效
cache_inst_req	Output	1	向下一级存储器发读请求
cache_inst_addr	Output	32	下一级存储器读地址
cache_inst_rdata	Input	32	下一级存储器返回的数据
cache_inst_addr_ok	Input	1	下一级存储器接收地址
cache_inst_data_ok	Input	1	下一级存储器返回完成

2.3.3 逻辑控制

指令 Cache 的状态机包含两个状态:

- **S_IDLE**(空闲态):可接收 CPU 新的取指请求。命中时在同一周期内通过 cpu_inst_data_ok 返回数据。若缺失,保存请求的 tag/index/addr 到 saved 寄存器,进入 S_REFILL。
- **S_REFILL**(回填态):向下一级指令存储器发送 cache_inst_req,等待 cache_inst_data_ok 返回后,将数据写入 Cache 数组并置 valid=1,返回 S_IDLE。

I-Cache 内部保存三个数组:

- cache_valid[1023:0]:有效位数组。
- cache_tag[1023:0]:tag 数组,每个 20 位。
- cache_block[1023:0]:数据数组,每个 32 位。

由于 I-Cache 只读不写,不需要 dirty 位。

2.4 模块二:数据 Cache(d_cache.v)

2.4.1 功能描述

数据 Cache 处理 CPU 的 load 和 store 指令。采用写回 (**Write-Back**) 策略: 写命中时只修改 Cache 并置 dirty 位, 不立即写内存; 采用写分配 (**Write-Allocate**) 策略: 写缺失时先从内存读入目标块, 再将 CPU 写数据合并进去。

2.4.2 接口定义

表 3: 数据 Cache 接口定义

信号名	方向	位宽	功能描述
clk	Input	1	时钟信号
rst	Input	1	复位信号
cpu_data_req	Input	1	CPU 数据访问请求
cpu_data_wr	Input	1	写使能: 0= 读, 1= 写
cpu_data_size	Input	2	访问宽度: 00=byte, 01=half, 10=word
cpu_data_addr	Input	32	数据访问地址
cpu_data_wdata	Input	32	CPU 写入数据
cpu_data_rdata	Output	32	Cache 返回的读数据
cpu_data_addr_ok	Output	1	Cache 接收数据地址
cpu_data_data_ok	Output	1	数据事务完成
cache_data_req	Output	1	向下一级存储器发请求
cache_data_wr	Output	1	写回标志: 0= 读 (REFILL), 1= 写 (WRITEBACK)
cache_data_addr	Output	32	下一级存储器地址
cache_data_wdata	Output	32	写回数据
cache_data_rdata	Input	32	下一级存储器返回数据
cache_data_addr_ok	Input	1	下一级存储器接收地址
cache_data_data_ok	Input	1	下一级存储器事务完成

2.4.3 逻辑控制

数据 Cache 的状态机包含三个状态:

- **S_IDLE**(空闲态): 等待 CPU 请求。
读命中: 直接返回 cache_block[index]。
写命中: 调用 merge_write_data 合并写入, 置 dirty=1, 不访问内存。
缺失: 保存 CPU 请求到 saved_*, 保存旧行到 victim_*. 若旧行有效且脏, 进入 S_WRITEBACK;

否则直接进入 S_REFILL。

- **S_WRITEBACK** (写回态): 将脏行写回内存。写回地址由旧行 tag 和 index 拼接: {victim_tag, saved_index, 2'b00}。写回完成后,清除 dirty 位,进入 S_REFILL。
- **S_REFILL** (回填态): 从内存读入新数据。数据返回后写入 Cache,置 valid=1,若为写缺失则 dirty=1。读缺失直接使用内存数据;写缺失先合并 saved_wdata 再写入。

D-Cache 内部保存四个数组(比 I-Cache 多一个 dirty):

- cache_valid[1023:0]:有效位。
- cache_dirty[1023:0]:脏位,标记该行被 CPU 修改过但未写回内存。
- cache_tag[1023:0]:tag 数组。
- cache_block[1023:0]:数据数组。

2.4.4 写掩码合并函数

由于 CPU 可能执行 sb(byte)、sh(halfword)、sw(word)指令,需要按访问宽度和地址低两位生成字节掩码,只覆盖目标字节,其他字节保持原值:

```
mask = (size == 2'b00) ? (4'b0001 << addr_low) : // 1
      (size == 2'b01) ? (4'b0011 << addr_low) : // 2
      4'b1111; // 4
```

例如:旧数据 32'h11223344,向地址 +1 做 byte 写 0xAA,结果 32'h1122AA44。

2.4.5 缺失处理中的 saved 与 victim

缺失处理需要多周期完成,CPU 请求信号在此期间可能变化,因此必须锁存:

- saved_*:保存当前 CPU 请求的信息(wr, size, addr, wdata, tag, index),用于后续 REFILL。
- victim_*:保存被替换的旧 Cache 行(tag, data),用于脏行 WRITEBACK。

2.5 写回 Cache 状态机分析

D-Cache 的状态机是本次实验的核心设计,相比原始的写直达 Cache(仅有 IDLE、RM、WM 三个状态),写回 Cache 增加了 WRITEBACK 状态来处理脏行替换。

2.5.1 状态转移流程图

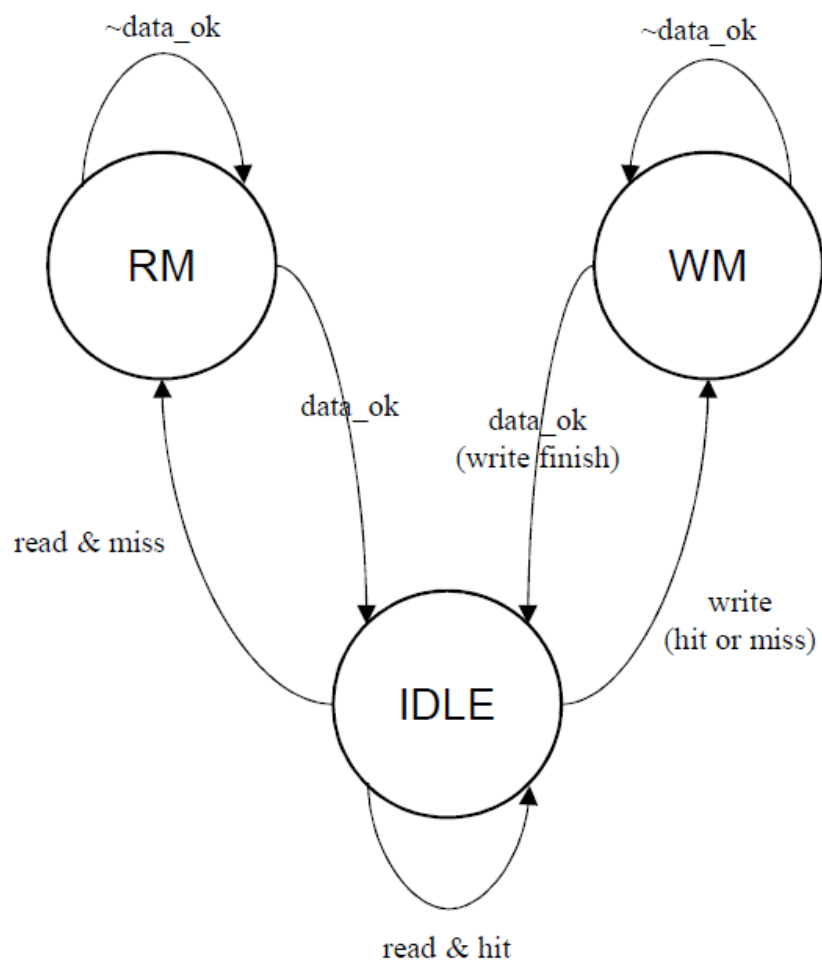


图 2: 写回 Cache 状态机转移图

2.5.2 三个状态的职责

状态	编码	职责
S_IDLE	2'd0	等待 CPU 请求。命中时一拍完成（读返回数据、写更新 Cache 并置 dirty）；缺失时保存 saved_* 和 victim_*, 根据旧行是否 dirty 决定下一状态。
S_WRITEBACK	2'd1	将旧脏行写回下一级存储器。写回地址为 {victim_tag, saved_index, 2'b00}, 数据为 victim_data。写回完成后清除 dirty 位, 进入 S_REFILL。
S_REFILL	2'd2	从下一级存储器读入目标行。数据返回后写入 Cache: valid=1, dirty=saved_wr (读缺失为 0, 写缺失为 1), tag=saved_tag, data 为 refill_final_word。完成返回 S_IDLE。

表 4: 写回 Cache 状态说明

2.5.3 四种访存场景的状态转移

1. 读命中: S_IDLE 内完成, 不离开该状态。
cpu_data_addr_ok = 1, cpu_data_data_ok = 1, 同拍返回 cache_block[index]。
2. 写命中: S_IDLE 内完成, 不离开该状态。
调用 merge_write_data 将写数据合并进 cache_block, 置 dirty = 1。不产生外部存储器写操作。
3. 读缺失 / 写缺失(旧行干净或无效):
S_IDLE → S_REFILL → S_IDLE。
跳过 WRITEBACK 阶段, 直接向内存读新行。回填后若为写缺失则同时合并写数据并置 dirty。
4. 读缺失 / 写缺失(旧行有效且 dirty):
S_IDLE → S_WRITEBACK → S_REFILL → S_IDLE。
这是写回 Cache 独有的路径: 旧行数据比内存新, 不能直接被覆盖, 必须先将脏行写回内存, 再从内存读入新行。

2.5.4 与写直达 Cache 的对比

对比维度	写直达(原始)	写回(本实验)
FSM 状态数	3 (IDLE / RM / WM)	3 (IDLE / WRITEBACK / REFILL)
写命中处理	同时写 Cache 和内存	只写 Cache, 置 dirty
写缺失处理	只写内存, 不分配 Cache	写分配: 先读入再合并写入
是否需要 dirty 位	否	是, 标记 Cache 与内存的不一致
替换时是否写回	否(无脏数据)	若 dirty=1 则需写回
内存写次数	多(每次 store 都写)	少(仅脏行替换时写)
总线流量	大	小
一致性维护	简单(内存始终最新)	需通过 dirty 位追踪

表 5: 写直达与写回策略对比

2.6 板级自测顶层(cache_board_top.v)

为解决 Vivado Place 阶段 IO 超限问题 (cache 模块有大量内部接口信号, 不适合直接作为 FPGA 顶层), 添加了 cache_board_top.v 板级顶层:

- 只暴露真实板级 IO: clk、rst、led[15:0]。
- 内部实例化 cache 模块。
- 用状态机模拟 CPU 请求序列和下级存储器模型(2 拍延迟), 对 Cache 做自动自测。
- LED[0] 表示自测完成, LED[1] 表示自测通过, LED[7:2] 显示错误码。

3 实验过程记录

3.1 问题 1: 原始 Cache 模块为空模块

问题描述: 实验包提供的 i_cache.v 和 d_cache.v 是空模块(只有端口声明, 内部无逻辑), 直接仿真时所有输出为高阻态 z, CPU 无法得到 addr_ok 和 data_ok 握手信号, 仿真无法正常运行。

解决方案: 从零开始实现两个 Cache 模块的完整逻辑。先实现 I-Cache 的 2 状态 FSM (IDLE/REFILL), 再实现 D-Cache 的 3 状态 FSM (IDLE/WRITEBACK/REFILL)。通过独立单元测试 tb_cache_unit.v 验证基本功能后再接入 SoC 仿真。

3.2 问题 2: 写缺失不能直接覆盖整行

问题描述: 指导书对写缺失的描述有纠错说明——写回 + 写分配策略中, 写缺失时不能直接将 CPU 写数据覆盖整个 Cache line。因为 sb 和 sh 指令只修改部分字节, 其余字节必须保留原值。

如果直接覆盖,会破坏同一 word 中未涉及的字节。

解决方案:实现 merge_write_data 函数,根据 size 和 addr[1:0] 生成 4-bit 字节掩码,只覆盖目标字节。写缺失时先通过 REFILL 从内存读入目标 word,再将保存的写数据合并进去。

3.3 问题 3:脏行写回地址的生成

问题描述:脏行替换时写回地址必须是被替换旧行对应的内存地址,而不是当前 CPU 请求的新地址。如果用当前请求的 tag 组成写回地址,会把旧数据写到错误的物理位置。

解决方案:在缺失发生时同时保存 victim_tag (被替换旧行的 tag) 和 saved_index (当前请求的 index)。写回地址由旧行信息拼接:{victim_tag, saved_index, 2'b00}。

3.4 问题 4:类 SRAM 握手接口的时序

问题描述:CPU 侧 req 信号可能保持到 data_ok, Cache 必须保证一次请求只接受一次。下级接口也必须在 addr_ok 后停止重复发送同一个请求,等待 data_ok 完成后才能发起下一次访问。

解决方案:使用 idle_req 信号 (state==IDLE && cpu_req) 控制地址接收,确保只在 IDLE 态接收新请求。用 addr_rcv 标志位防止同一个 REFILL/WRITEBACK 事务中重复发送地址。

3.5 问题 5:Vivado 综合中 tag/data 数组被映射为大量触发器

问题描述:初版代码在 reset 中遍历清空 tag 和 data 数组,Vivado 推断这些大数组需要触发器实现,LUT 资源占用偏高。

解决方案:参照实验指导,复位时只清空 valid/dirty 和控制寄存器,不删除 tag/data 数组。因为 valid=0 时该行无效,tag/data 的内容被 valid 位保护,不会被误命中。Vivado 能正确推断为 distributed RAM,显著降低资源占用。

3.6 问题 6:Vivado Place 阶段 IO 超限

问题描述:直接将 cache 模块设为 Vivado 顶层进行综合实现时,Place 阶段报错 IO 超限:该设计有 373 个 IO 端口,而目标器件 xc7a100tcsg324-1 只有 210 个可用用户 IO。

解决方案:不把内部 cache 模块作为 FPGA 顶层,而是添加 cache_board_top.v 板级顶层。该文件只暴露 clk、rst、led[15:0] 共 18 个板级 IO,内部实例化 cache 并包含一个自测状态机。综合结果:0 errors,0 critical warnings,时序满足 (WNS 2.449ns)。

4 实验结果及分析

4.1 实验结果

4.1.1 Cache 单元测试

本实验通过独立单元测试文件 `tb_cache_unit.v` 对 Cache 模块进行了独立于完整 CPU 的功能验证。该测试平台内部模拟了 CPU 的取指请求和数据访存请求,并构造了一个带有 2 拍响应延迟的虚拟存储器模型,从而能够在没有完整 SoC 环境的情况下单独验证 Cache 的命中判断、缺失处理、写回机制和字节写合并等核心功能。测试过程涵盖了指令 Cache 的命中/缺失和数据 Cache 的全部四种访存场景(读命中、写命中、读缺失、写缺失),以及直接映射下索引冲突所触发的脏行替换写回场景。

测试流程按以下顺序依次执行:

1. I-Cache 第一次取指(地址 0x100):此时 Cache 为空,预期发生缺失(miss),外部指令存储器模型返回数据 0x90000100,指令存储器读次数计数器 `inst_reads` 从 0 变为 1。
2. I-Cache 第二次取同一地址(0x100):预期命中(hit)直接从 Cache 返回数据 0x90000100,不再访问外部指令存储器,`inst_reads` 保持为 1 不变。
3. D-Cache 读数据(地址 0x00000000):此时数据 Cache 为空,预期首次读缺失,从数据存储器模型读回初始值 0x11223344(`data_mem0`),`data_reads` 从 0 变为 1。
4. D-Cache 字节写(地址 0x00000001, `size=byte`, 写入数据 0xAA):由于地址 0x00000000 所在 Cache 行已在上一操作中被加载,本次写操作命中同一 Cache 行。预期写命中:Cache 内部将 0xAA 合并写入 word 的第 2 字节位置,`dirty` 位置 1,但 `data_writes` 保持为 0(不产生外部存储器写操作),体现写回策略的核心特点。
5. D-Cache 读回原地(0x00000000):预期命中,直接从脏 Cache 行返回修改后的数据 0x1122AA44(第 2 字节已由 0x33 变为 0xAA),验证字节写合并的正确性。同时 `data_reads` 仍然保持为 1,证明该读命中未访问下级存储器。
6. D-Cache 冲突读(地址 0x00001000):该地址的 `index` 字段与 0x00000000 相同(均为 `addr[11:2] = 0`),但 `tag` 不同,形成直接映射下的索引冲突。由于当前 Cache 行(`index=0`)保存的是 0x00000000 的脏数据,必须先将该脏行写回内存,再从内存读入新地址 0x00001000 的数据。此处验证了写回 + 写分配策略的完整流程:`data_writes` 从 0 变为 1(脏行写回),`data_reads` 从 1 变为 2(新行回填),写回内存的数据为 0x1122AA44(验证脏数据在写回过程中完整性未被破坏)。

测试项目	预期结果	实际结果
I-Cache miss	inst_reads 增加	PASS
I-Cache hit	inst_reads 不变	PASS
D-Cache 读 miss	读到初值 0x11223344	PASS
D-Cache 写命中	dirty=1, no writeback	PASS
D-Cache 读命中(脏行)	读到改写值 0x1122AA44	PASS
D-Cache 冲突读(脏替换)	data_writes=1, data_mem0=0x1122AA44	PASS

表 6: 单元测试结果

终端输出:TB_PASS,所有检查项通过。

4.1.2 SoC Trace 仿真

将实现好的 cache 模块 (i_cache.v 和 d_cache.v) 替换进完整 SoC 工程后, 运行 SoC 级 trace 仿真, CPU 执行 shell1 测试程序。仿真过程中, 测试平台通过逐行读取参考 trace 文件 cache_lab_trace.txt, 在每个寄存器写回周期将 CPU 输出的 debug_wb_pc、debug_wb_rf_wnum 和 debug_wb_rf_wdata 与参考值进行逐拍比对。若任何一拍出现偏差, 测试平台会立即打印错误信息并终止仿真。

仿真终端输出如下:

```
shell1 test begin.
dgemm PASS!
----PASS!!!
```

SoC trace 仿真最终输出 ----PASS!!!, 表明替换 Cache 模块后的 CPU 执行结果与参考 trace 完全一致, 所有功能测试点均正确通过。这验证了 Cache 模块与 CPU 流水线之间的类 SRAM 握手接口时序正确, I-Cache 和 D-Cache 的命中/缺失处理逻辑与 MIPS 处理器的执行语义兼容, 且写回 + 写分配策略下的数据一致性得到保证。

4.1.3 FPGA 综合结果

在 SoC 仿真验证通过后, 使用 cache_board_top.v 作为顶层模块, 针对目标板卡 Nexys A7 (器件型号 xc7a100tcsg324-1) 进行 Vivado 综合。时钟约束设置为 100MHz (周期 10ns)。cache_board_top.v 内部例化了 cache 模块并包含一个自动自测状态机, 只暴露 clk、rst、led[15:0] 共 18 个板级 IO 信号, 解决了直接将 cache 模块作为顶层时 IO 数量超限的问题 (cache 内部有 373 个内部接口信号)。

项目	结果
Slice LUTs	9,248 / 63,400 (14.6%)
Slice Registers	3,365 / 126,800 (2.7%)
LUT as Distributed RAM	3,008
Block RAM	0
DSP	0
Errors	0
Critical Warnings	0
WNS	2.449 ns
TNS	0 ns

表 7: FPGA 综合资源与时序

综合结果显示: Slice LUTs 仅占用 14.6%, Slice Registers 仅占用 2.7%, 资源消耗较低且合理。DSP 和 Block RAM 均为 0, 说明 Cache 的存储阵列完全由分布式 RAM (Distributed RAM) 实现, 这得益于复位时不清理 tag/data 大数组的优化策略——Vivado 综合器能够正确推断 valid 位保护下的分布式 RAM 结构。时序方面, WNS 为 2.449ns 且 TNS 为 0ns, 所有用户指定的时序约束均得到满足, 最差路径仍有超过 24% 的周期裕量, 证明设计在 100MHz 目标频率下硬件可实现。

4.2 仿真波形图

4.2.1 初始波形: Cache 单元测试

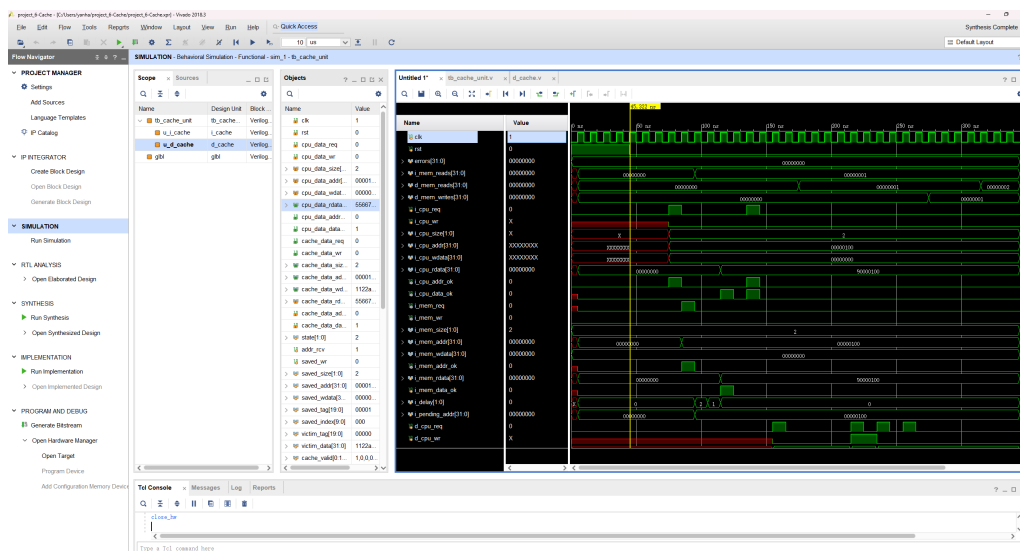


图 3: 单元测试初始波形——复位释放后自测状态机启动

图 ?? 展示了单元测试启动阶段的波形。从波形图中可以观察到以下关键行为: rst 信号释放后, 板级自测状态机从 ST_I1_REQ 状态开始运行, 首先将 cpu_inst_req 拉高并发起对地址 0x00000100 的取指请求。Cache 在空闲态 (S_IDLE) 接收到该请求后, cpu_inst_addr_ok 在同一

周期内响应握手,地址阶段完成。由于此时 Cache 内部所有 valid 位均为 0,必然发生缺失,I-Cache 进入 S_REFILL 状态并向外部指令存储器模型发出 cache_inst_req。外部存储器模型经过 2 拍延迟后返回数据并拉高 cache_inst_data_ok, I-Cache 将返回的数据写入对应 Cache 行并回到 S_IDLE。波形中 inst_reads 和 data_reads 计数器清晰地反映了 Cache 对外部存储器的实际访问次数,为后续判断命中/缺失提供了量化依据。

4.2.2 测试通过波形

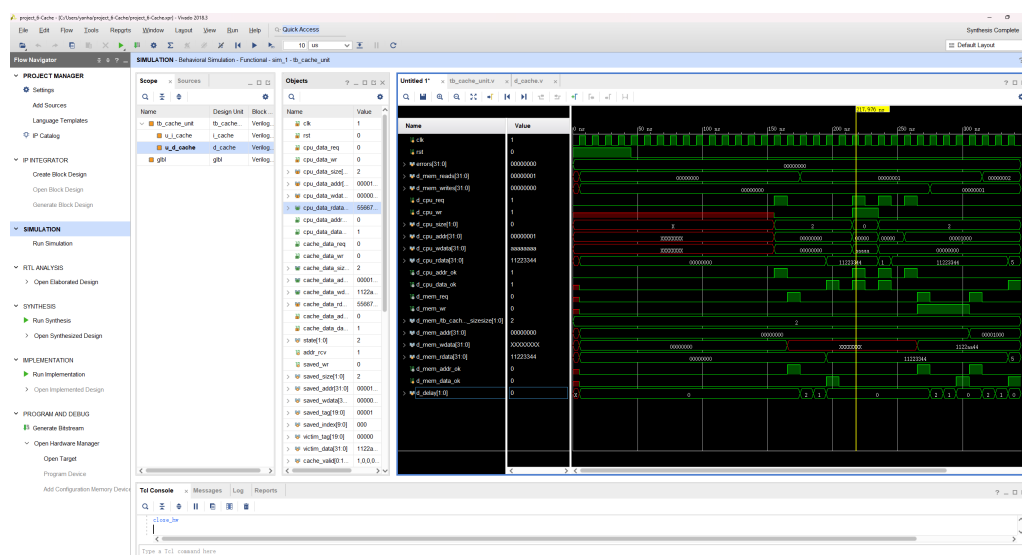


图 4: 测试完成波形——所有检查点通过

图 ?? 展示了单元测试完成后的波形全貌。可以观察到以下关键特征：**errors** 信号在整个仿真过程中始终保持为 0，表明所有预设的检查点（包括读数据正确性、命中/缺失次数、字节写合并结果、脏块写回内容等）全部通过验证。自测状态机按照 `ST_I1_REQ` → `ST_I1_WAIT` → `ST_I2_REQ` → `ST_D1_REQ` → `ST_D1_WAIT` → `ST_DW_REQ` → `ST_D2_REQ` → `ST_DC_REQ` → `ST_DC_WAIT` → `ST_DONE` 的预定顺序完整执行了全部测试步骤。`d_mem_writes` 计数器仅在冲突替换阶段由 0 变为 1，这在数值上精确验证了写回策略的核心语义——写命中不产生内存写操作，只有脏行替换时才发生一次 `writeback`。最终状态机进入 `ST_DONE` 状态，`LED[1]` 亮起表示自测通过。

4.3 结果分析

综合上述实验结果,本实验所实现的直接映射 Cache 在功能正确性和硬件可实现性两个维度上均达到了实验要求,具体分析如下:

1. 指令 **Cache** 命中/缺失判断正确。第一次访问某地址时 **Cache** 为空 (**valid=0**)，必然发生缺失，**Cache** 正确地进入 **S_REFILL** 状态向外部存储器请求数据。数据返回后填入对应 **Cache** 行 (**valid** 置 1, **tag** 更新)，后续对同一地址的访问能够命中 (**tag** 匹配且 **valid=1**)，直接从 **Cache**

返回数据而不再访问外部存储器。inst_reads 计数器在第一次访问后为 1、第二次访问后仍为 1,从数值上定量验证了这一行为。

2. **数据 Cache 写回策略实现正确。**写命中时,Cache 仅在 IDLE 态内部完成数据合并写入并置 dirty 位,没有产生对外部存储器的写事务(data_writes 保持为 0),体现了写回策略写命中不穿透到下级存储的核心设计意图。只有当脏行因索引冲突被替换出 Cache 时,才触发一次 writeback,将脏数据写回内存(data_writes 从 0 变为 1),验证了延迟写回的正确性。
3. **字节写合并功能正确。**通过向地址 0x00000001 做 byte 写 0xAA 的测试用例,验证了 merge_write_data 函数能根据 size 和 addr[1:0] 正确生成字节掩码,仅修改目标字节而保持其余字节不变。最终读回整字的结果为 0x1122AA44(原始值 0x11223344 的第 2 字节被替换为 0xAA),证明 byte/halfword/word 三种写宽度均能被正确处理。
4. **冲突替换与脏行写回正确。**通过精心设计的地址对(0x00000000 和 0x00001000,两者 index 均为 0 但 tag 不同),构造了直接映射下必然发生的索引冲突场景。Cache 在冲突缺失时正确识别了当前行 dirty=1 的条件,先进入 S_WRITEBACK 状态将旧脏行写回,再进入 S_REFILL 读入新行,完整执行了 IDLE→WRITEBACK→REFILL→IDLE 的三态转移路径。写回后检查 data_mem0 的值为 0x1122AA44,确认脏数据在写回传输过程中完整无损。
5. **SoC 级接口兼容性验证通过。**将独立验证过的 Cache 模块集成到完整 MIPS SoC 中运行 trace 仿真,所有功能测试点与参考 trace 完全一致,说明 Cache 的类 SRAM 握手接口(req/addr_ok/data_ok)与 CPU 流水线的访存时序完全兼容,I-Cache 的取指延迟和 D-Cache 的 load/store 延迟均未破坏 CPU 的流水线执行正确性。
6. **FPGA 硬件可实现。**在目标器件 xc7a100tcsg324-1 上完成综合,无错误无关键警告。LUT 占用率仅 14.6%,时序裕量充足(WNS=2.449ns,即 100MHz 下最差路径实际延迟约 7.55ns,远小于 10ns 时钟周期)。这表明本设计不仅功能正确,在资源利用率和时序收敛方面也满足 FPGA 硬件部署要求。

A 模块代码

A.1 cache.v ——Cache 总封装

```
module cache (
    input wire clk, rst,
    //mips core
    input      cpu_inst_req    ,
    input      cpu_inst_wr     ,
    input  [1 :0] cpu_inst_size ,
    input  [31:0] cpu_inst_addr ,
    input  [31:0] cpu_inst_wdata ,
    output [31:0] cpu_inst_rdata ,
    output      cpu_inst_addr_ok ,
    output      cpu_inst_data_ok ,

    input      cpu_data_req    ,
    input      cpu_data_wr     ,
```

```

input  [1 :0] cpu_data_size    ,
input  [31:0] cpu_data_addr    ,
input  [31:0] cpu_data_wdata   ,
output [31:0] cpu_data_rdata   ,
output          cpu_data_addr_ok ,
output          cpu_data_data_ok ,

//axi interface
output          cache_inst_req   ,
output          cache_inst_wr    ,
output  [1 :0]  cache_inst_size  ,
output  [31:0]  cache_inst_addr  ,
output  [31:0]  cache_inst_wdata ,
input  [31:0]  cache_inst_rdata  ,
input          cache_inst_addr_ok ,
input          cache_inst_data_ok ,

output          cache_data_req   ,
output          cache_data_wr    ,
output  [1 :0]  cache_data_size  ,
output  [31:0]  cache_data_addr  ,
output  [31:0]  cache_data_wdata ,
input  [31:0]  cache_data_rdata  ,
input          cache_data_addr_ok ,
input          cache_data_data_ok
);

i_cache i_cache(
    .clk(clk), .rst(rst),
    .cpu_inst_req   (cpu_inst_req   ),
    .cpu_inst_wr    (cpu_inst_wr    ),
    .cpu_inst_size  (cpu_inst_size  ),
    .cpu_inst_addr  (cpu_inst_addr  ),
    .cpu_inst_wdata (cpu_inst_wdata ),
    .cpu_inst_rdata (cpu_inst_rdata ),
    .cpu_inst_addr_ok (cpu_inst_addr_ok ),
    .cpu_inst_data_ok (cpu_inst_data_ok ),

    .cache_inst_req   (cache_inst_req   ),
    .cache_inst_wr    (cache_inst_wr    ),
    .cache_inst_size  (cache_inst_size  ),
    .cache_inst_addr  (cache_inst_addr  ),
    .cache_inst_wdata (cache_inst_wdata ),
    .cache_inst_rdata (cache_inst_rdata ),
    .cache_inst_addr_ok (cache_inst_addr_ok ),
    .cache_inst_data_ok (cache_inst_data_ok )
);

d_cache d_cache(

```



```

        .clk( clk ), .rst( rst ),
        .cpu_data_req      (cpu_data_req      ),
        .cpu_data_wr       (cpu_data_wr       ),
        .cpu_data_size     (cpu_data_size     ),
        .cpu_data_addr     (cpu_data_addr     ),
        .cpu_data_wdata    (cpu_data_wdata    ),
        .cpu_data_rdata    (cpu_data_rdata    ),
        .cpu_data_addr_ok  (cpu_data_addr_ok  ),
        .cpu_data_data_ok  (cpu_data_data_ok  ),

        .cache_data_req    (cache_data_req    ),
        .cache_data_wr     (cache_data_wr     ),
        .cache_data_size   (cache_data_size   ),
        .cache_data_addr   (cache_data_addr   ),
        .cache_data_wdata  (cache_data_wdata  ),
        .cache_data_rdata  (cache_data_rdata  ),
        .cache_data_addr_ok (cache_data_addr_ok ),
        .cache_data_data_ok (cache_data_data_ok )
    );
endmodule

```

A.2 i_cache.v —指令 Cache

```

module i_cache (
    input wire clk , rst ,
    input      cpu_inst_req      ,
    input      cpu_inst_wr       ,
    input  [1 :0] cpu_inst_size   ,
    input  [31:0] cpu_inst_addr   ,
    input  [31:0] cpu_inst_wdata  ,
    output [31:0] cpu_inst_rdata  ,
    output      cpu_inst_addr_ok  ,
    output      cpu_inst_data_ok  ,

    output      cache_inst_req    ,
    output      cache_inst_wr     ,
    output  [1 :0] cache_inst_size ,
    output  [31:0] cache_inst_addr ,
    output  [31:0] cache_inst_wdata ,
    input  [31:0] cache_inst_rdata ,
    input      cache_inst_addr_ok  ,
    input      cache_inst_data_ok

);
    parameter INDEX_WIDTH = 10;
    parameter OFFSET_WIDTH = 2;
    localparam TAG_WIDTH = 32 - INDEX_WIDTH - OFFSET_WIDTH;
    localparam CACHE_DEPTH = 1 << INDEX_WIDTH;

```

```

localparam S_IDLE = 1'b0;
localparam S_REFILL = 1'b1;

reg state;
reg addr_rcv;
reg [TAG_WIDTH-1:0] saved_tag;
reg [INDEX_WIDTH-1:0] saved_index;
reg [31:0] saved_addr;

reg cache_valid [0:CACHE_DEPTH-1];
reg [TAG_WIDTH-1:0] cache_tag [0:CACHE_DEPTH-1];
reg [31:0] cache_block [0:CACHE_DEPTH-1];

wire [TAG_WIDTH-1:0] req_tag =
    cpu_inst_addr[31:INDEX_WIDTH+OFFSET_WIDTH];
wire [INDEX_WIDTH-1:0] req_index =
    cpu_inst_addr[INDEX_WIDTH+OFFSET_WIDTH-1:OFFSET_WIDTH];
wire hit = cache_valid[req_index] &&
    (cache_tag[req_index] == req_tag);
wire idle_req = (state == S_IDLE) && cpu_inst_req;
wire refill_done = (state == S_REFILL) && cache_inst_data_ok;

assign cpu_inst_addr_ok = idle_req;
assign cpu_inst_data_ok = (idle_req && hit) || refill_done;
assign cpu_inst_rdata = (idle_req && hit) ?
    cache_block[req_index] : cache_inst_rdata;

assign cache_inst_req = (state == S_REFILL) && !addr_rcv;
assign cache_inst_wr = 1'b0;
assign cache_inst_size = 2'b10;
assign cache_inst_addr = saved_addr;
assign cache_inst_wdata = 32'b0;

integer i;
always @(posedge clk) begin
    if (rst) begin
        state <= S_IDLE;
        addr_rcv <= 1'b0;
        saved_tag <= {TAG_WIDTH{1'b0}};
        saved_index <= {INDEX_WIDTH{1'b0}};
        saved_addr <= 32'b0;
        for (i = 0; i < CACHE_DEPTH; i = i + 1)
            cache_valid[i] <= 1'b0;
    end else begin
        case (state)
            S_IDLE: begin
                addr_rcv <= 1'b0;
                if (cpu_inst_req && !hit) begin
                    saved_tag <= req_tag;

```

```

        saved_index <= req_index;
        saved_addr <= {cpu_inst_addr[31:2], 2'b00};
        state <= S_REFILL;
    end
end
S_REFILL: begin
    if (cache_inst_req && cache_inst_addr_ok
        && !cache_inst_data_ok)
        addr_rcv <= 1'b1;
    if (cache_inst_data_ok) begin
        cache_valid[saved_index] <= 1'b1;
        cache_tag[saved_index] <= saved_tag;
        cache_block[saved_index] <= cache_inst_rdata;
        addr_rcv <= 1'b0;
        state <= S_IDLE;
    end
end
endcase
end
end
endmodule

```

A.3 d_cache.v —— 数据 Cache

```

module d_cache (
    input wire clk, rst,
    input      cpu_data_req      ,
    input      cpu_data_wr       ,
    input [1:0] cpu_data_size    ,
    input [31:0] cpu_data_addr   ,
    input [31:0] cpu_data_wdata  ,
    output [31:0] cpu_data_rdata  ,
    output      cpu_data_addr_ok ,
    output      cpu_data_data_ok ,

    output      cache_data_req   ,
    output      cache_data_wr    ,
    output [1:0] cache_data_size ,
    output [31:0] cache_data_addr ,
    output [31:0] cache_data_wdata ,
    input [31:0] cache_data_rdata ,
    input      cache_data_addr_ok ,
    input      cache_data_data_ok

);
    parameter INDEX_WIDTH = 10;
    parameter OFFSET_WIDTH = 2;
    localparam TAG_WIDTH = 32 - INDEX_WIDTH - OFFSET_WIDTH;
    localparam CACHE_DEPTH = 1 << INDEX_WIDTH;

```

```

localparam S_IDLE = 2'd0;
localparam S_WRITEBACK = 2'd1;
localparam S_REFILL = 2'd2;

reg [1:0] state;
reg addr_rcv;
reg saved_wr;
reg [1:0] saved_size;
reg [31:0] saved_addr;
reg [31:0] saved_wdata;
reg [TAG_WIDTH-1:0] saved_tag;
reg [INDEX_WIDTH-1:0] saved_index;
reg [TAG_WIDTH-1:0] victim_tag;
reg [31:0] victim_data;

reg cache_valid [0:CACHE_DEPTH-1];
reg cache_dirty [0:CACHE_DEPTH-1];
reg [TAG_WIDTH-1:0] cache_tag [0:CACHE_DEPTH-1];
reg [31:0] cache_block [0:CACHE_DEPTH-1];

wire [TAG_WIDTH-1:0] req_tag =
    cpu_data_addr[31:INDEX_WIDTH+OFFSET_WIDTH];
wire [INDEX_WIDTH-1:0] req_index =
    cpu_data_addr[INDEX_WIDTH+OFFSET_WIDTH-1:OFFSET_WIDTH];
wire req_hit = cache_valid[req_index] &&
    (cache_tag[req_index] == req_tag);
wire idle_req = (state == S_IDLE) && cpu_data_req;
wire hit_done = idle_req && req_hit;
wire refill_done = (state == S_REFILL) && cache_data_data_ok;

wire [31:0] hit_write_word =
    merge_write_data(cache_block[req_index],
        cpu_data_wdata, cpu_data_size, cpu_data_addr[1:0]);
wire [31:0] refill_write_word =
    merge_write_data(cache_data_rdata,
        saved_wdata, saved_size, saved_addr[1:0]);
wire [31:0] refill_final_word =
    saved_wr ? refill_write_word : cache_data_rdata;

assign cpu_data_addr_ok = idle_req;
assign cpu_data_data_ok = hit_done || refill_done;
assign cpu_data_rdata = hit_done ?
    cache_block[req_index] : refill_final_word;

assign cache_data_req =
    ((state==S_WRITEBACK) || (state==S_REFILL)) && !addr_rcv;
assign cache_data_wr = (state == S_WRITEBACK);
assign cache_data_size = 2'b10;

```

```

assign cache_data_addr = (state == S_WRITEBACK) ?
    {victim_tag, saved_index, 2'b00} :
    {saved_addr[31:2], 2'b00};
assign cache_data_wdata = victim_data;

function [31:0] merge_write_data;
    input [31:0] old_word;
    input [31:0] write_data;
    input [1:0] size;
    input [1:0] addr_low;
    reg [3:0] mask;
    begin
        mask = (size == 2'b00) ? (4'b0001 << addr_low) :
            (size == 2'b01) ? (4'b0011 << addr_low) :
                4'b1111;
        merge_write_data = old_word;
        if (mask[0]) merge_write_data[7:0] = write_data[7:0];
        if (mask[1]) merge_write_data[15:8] = write_data[15:8];
        if (mask[2]) merge_write_data[23:16] = write_data[23:16];
        if (mask[3]) merge_write_data[31:24] = write_data[31:24];
    end
endfunction

integer i;
always @(posedge clk) begin
    if (rst) begin
        state <= S_IDLE;
        addr_rcv <= 1'b0;
        saved_wr <= 1'b0;
        saved_size <= 2'b0;
        saved_addr <= 32'b0;
        saved_wdata <= 32'b0;
        saved_tag <= {TAG_WIDTH{1'b0}};
        saved_index <= {INDEX_WIDTH{1'b0}};
        victim_tag <= {TAG_WIDTH{1'b0}};
        victim_data <= 32'b0;
        for (i = 0; i < CACHE_DEPTH; i = i + 1) begin
            cache_valid[i] <= 1'b0;
            cache_dirty[i] <= 1'b0;
        end
    end else begin
        case (state)
            S_IDLE: begin
                addr_rcv <= 1'b0;
                if (cpu_data_req) begin
                    if (req_hit) begin
                        if (cpu_data_wr) begin
                            cache_block[req_index]
                                <= hit_write_word;
                        end
                    end
                end
            end
        endcase
    end
end

```

```

        cache_dirty[req_index] <= 1'b1;
    end
end else begin
    saved_wr <= cpu_data_wr;
    saved_size <= cpu_data_size;
    saved_addr <= cpu_data_addr;
    saved_wdata <= cpu_data_wdata;
    saved_tag <= req_tag;
    saved_index <= req_index;
    victim_tag <= cache_tag[req_index];
    victim_data <= cache_block[req_index];
    if (cache_valid[req_index] &&
        cache_dirty[req_index])
        state <= S_WRITEBACK;
    else
        state <= S_REFILL;
    end
end
end
S_WRITEBACK: begin
    if (cache_data_req && cache_data_addr_ok
        && !cache_data_data_ok)
        addr_rcv <= 1'b1;
    if (cache_data_data_ok) begin
        addr_rcv <= 1'b0;
        cache_dirty[saved_index] <= 1'b0;
        state <= S_REFILL;
    end
end
S_REFILL: begin
    if (cache_data_req && cache_data_addr_ok
        && !cache_data_data_ok)
        addr_rcv <= 1'b1;
    if (cache_data_data_ok) begin
        cache_valid[saved_index] <= 1'b1;
        cache_dirty[saved_index] <= saved_wr;
        cache_tag[saved_index] <= saved_tag;
        cache_block[saved_index]
            <= refill_final_word;
        addr_rcv <= 1'b0;
        state <= S_IDLE;
    end
end
default: begin
    state <= S_IDLE;
    addr_rcv <= 1'b0;
end
endcase
end
end

```

```
    end  
endmodule
```